



PDF Download
3672608.3707723.pdf
18 March 2026
Total Citations: 0
Total Downloads: 630

 Latest updates: <https://dl.acm.org/doi/10.1145/3672608.3707723>

RESEARCH-ARTICLE

Elastic Vertical Memory Management for Container-based Stateful Applications in Kubernetes

TAESHIN KANG, Korea University, Seoul, South Korea

HEONCHANG YU, Korea University, Seoul, South Korea

Open Access Support provided by:

Korea University

Published: 14 May 2025

[Citation in BibTeX format](#)

SAC '25: 40th ACM/SIGAPP Symposium
on Applied Computing
March 31 - April 4, 2025
Catania, Italy

Conference Sponsors:
SIGAPP

Elastic Vertical Memory Management for Container-based Stateful Applications in Kubernetes

Taeshin Kang
Dept. of Computer Science and Engineering
Korea University
Seoul, Republic of Korea
teri98@korea.ac.kr

Heonchang Yu
Dept. of Computer Science and Engineering
Korea University
Seoul, Republic of Korea
yuhc@korea.ac.kr

ABSTRACT

Cloud service providers offer horizontal auto-scaling to prevent wasting resources by overprovisioning containers. However, this approach struggles to respond immediately to unexpected spikes in resource usage and has limitations in optimizing resource utilization. Vertical auto-scaling overcomes these limitations. Vertical elasticity has become increasingly important in recent years to provide instant resource elasticity for stateful and real-time critical applications. Nevertheless, vertical elasticity is not actively researched and implemented due to limitations in achieving it. This paper proposes Elastic Vertical Memory Management (EVMM) for stateful applications in the Kubernetes environment, which is the de facto standard in the container orchestration space. We compared EVMM to an existing Kubernetes environment without vertical elasticity and found that it reduced the total execution time of container workflows by up to 35% and memory resource utilization on nodes improved by 1.5x.

CCS CONCEPTS

• **Computer systems organization** → **Architectures** → **Distributed architectures** → **Cloud computing**

KEYWORDS

Auto-scaling, Vertical auto-scaling, Memory Elasticity, Kubernetes, Container

ACM Reference format:

Taeshin Kang, and Heonchang Yu. 2025. Elastic Vertical Memory Management for Container-based Stateful Applications in Kubernetes. In *The 40th ACM/SIGAPP Symposium on Applied Computing (SAC '25)*, March 31-April 4, 2025, Sicily, Italy. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3672608.3707723>

1 INTRODUCTION

Container technology has become widely used in cloud environments due to its portability and lower overhead compared to traditional virtualization technologies [1, 2]. Cloud service providers offer predefined, fixed-sized containers to ensure adequate performance isolation for users. However, due to this

constraint, cloud users often over-provision containers to avoid resource shortages [3]. This practice leads to the underutilization of system resources (e.g., CPU, memory) in the cloud [4]. For instance, an analysis of jobs executed on one of Google's clusters over the course of a month found that the average memory utilization of the cluster was only 50% [5, 6]. Since it is challenging for cloud users to efficiently configure resources within containers, cloud service providers offer container auto-scaling. Auto-scaling allows for flexible allocation(scale-up) or reclamation(scale-down) of resources within a container, providing resource elasticity. Resource elasticity can be broadly categorized into two types: horizontal elasticity and vertical elasticity. Horizontal elasticity adjusts the number of containers handling a job, while vertical elasticity adjusts the amount of resources allocated to each container. Cloud service providers often use a horizontal auto-scaling approach. However, horizontal auto-scaling has a relatively high overhead when launching new instances, making it difficult to respond immediately to unexpected spikes in resource usage. Additionally, it is not suitable for applications that cannot be replicated or decomposed into smaller, independent units [7, 8]. Vertical auto-scaling can be used to address the following limitations. Recently, there has been a growing need for research on vertical auto-scaling to manage stateful applications, such as scientific workflows and machine learning jobs, and to provide instant resource elasticity for real-time, critical applications [9, 10]. However, container vertical elasticity has not been actively researched [11], and related industry solutions are difficult to apply in real-world applications. For example, the Vertical Pod Autoscaler (VPA) [12], supported by Kubernetes [13], the de facto standard for container orchestration that provides unified management and control of containers, restarts containers when a scale-up in resources occurs, which means that tasks must be reinitiated from the beginning [14].

Despite the growing importance of vertical elasticity, it is not actively researched and implemented due to the limitations in achieving it. Each system has a physical limit on its resources, so it is not possible to scale CPU or memory indefinitely [15]. In particular, achieving memory resource elasticity is more difficult than achieving CPU resource elasticity because CPU resources can be shared among processes with some degree of flexibility, while memory resources cannot. To overcome the constraints of physical memory size, swap memory can be considered. Swap memory prevents applications from running out of memory and causing OOM(out-of-memory) errors [16]. However, swap memory allocates some of the disk space to memory, which reduces the performance of the application and system [17]. Furthermore, because Kubernetes does not support swap memory by default, containers that request memory allocations exceeding the system's physical memory will cause an OOM error and be restarted [18]. Therefore, managing vertical memory elasticity of applications in a Kubernetes environment is a challenging problem.



This work is licensed under a Creative Commons Attribution International 4.0 License.

SAC '25, March 31-April 4, 2025, Catania, Italy
© 2025 Copyright is held by the owner/author(s).
ACM ISBN 979-8-4007-0629-5/25/03.
<https://doi.org/10.1145/3672608.3707723>

We propose Elastic Vertical Memory Management (EVMM) for stateful applications in Kubernetes environments. To the best of our knowledge, there has been little research focusing on memory elasticity in Kubernetes environments, particularly when swap memory is disabled. We begin by modeling the priority of each task and then perform a scale-up on the tasks based on these priorities. Next, we pause and checkpoint tasks by limiting the CPU usage of those that fail to scale-up. If memory pressure does not improve, the paused tasks are terminated in order of decreasing priority. Conversely, if memory pressure improves, the stopped tasks continue from the checkpoint. When comparing EVMM to a Kubernetes environment without vertical elasticity, we found that the total execution time of container workflows was reduced by up to 35%, and container restarts were reduced by up to 70x compared to an environment with excessive memory oversubscription.

This paper is organized as follows. Section 2 introduces research on vertical scaling and memory elasticity relevant to this study. Section 3 defines the scope of the problem addressed in the paper. In Section 4, we mathematically model metrics such as resources, penalties, and rewards for task priorities and the magnitude of scaling. Section 5 presents an algorithmic representation and description of EVMM. Section 6 details the implementation of EVMM. In Section 7, we evaluate the effectiveness of EVMM. Finally, Section 8 analyzes the contributions of this work and outlines future research directions.

2 RELATED WORK

Although not as numerous as horizontal scaling, the following container-based vertical scaling or memory elasticity research has been proposed. ElasticDocker [19] is the first paper to propose container-based vertical scaling, and it uses the MAPE-K principle to vertically scale Docker-based containers. If the host machine does not have enough resources, it migrates containers to another machine in real time based on examined metrics. Comparing ElasticDocker to Kubernetes HPA, ElasticDocker achieved a runtime reduction of about 37%. RUBAS [20] is a technique for dynamically adjusting the allocation of containers running in a Kubernetes cluster. If the machine to which the application is assigned has enough resources, it scales the container vertically; if it runs out of resources, it performs a disruptive container migration to another machine. Compared to Kubernetes VPA, RUBAS improved CPU and memory utilization in clusters by 10% and reduced runtime by 15%. However, these early studies used a rather simplistic approach to resource scaling, applying the same scaling process per job, and did not account for job prioritization.

Several approaches have since been proposed that utilize vertical scaling. Wang et al. proposed DYVERSE [21], which supports dynamic vertical scaling in multi-tenant edge environments. They propose dynamic prioritization approach that considers workloads, communities, and systems to enable dynamic vertical scaling. Buchaca et al. [22] propose a predictive vertical auto-scaling policy that uses a time-window multi-layer perceptron (MLP) to predict the stages of different types of containers and dynamically resize containers accordingly. Choi et al. [23] utilized the vertical elasticity feature provided by Docker to dynamically adjust computing resources to improve system throughput in big data-based applications. Kwan et al. proposed HyScale[7], which utilizes both horizontal and vertical scaling of containers in a Docker-based environment. However, these works did not pay attention to memory elasticity.

Recent research has explored vertical elasticity in Kubernetes environments. Zhao et al. [24] proposed a dynamic CPU allocation technique tailored for serverless workloads, while SlackAuto [25]

introduced a slack-aware CPU vertical scaling mechanism to enhance efficiency and performance in serverless applications. Similarly, CaaSPER [26] dynamically adjusted CPU resources to minimize slack and throttling, ensuring optimal usage. However, these studies overlooked memory elasticity and did not address the container restart issues of Kubernetes VPA.

Pufferish [27], on the other hand, focuses on memory elasticity and shows how container-based lightweight virtualization opens up opportunities to achieve memory elasticity. They implement an elastic memory manager that leverages containers to flexibly allocate memory for tasks. Pufferish is implemented in Apache Yarn and has been shown to prevent OOM errors and improve memory utilization. However, pufferish is based on environments where swap memory is enabled, making it difficult to apply to environments where swap memory is disabled.

3 PROBLEM MODELING

3.1 Model Description

Table 1 shows a description of the terms and abbreviations mentioned in this paper. Let $N = \{n_1, n_2, n_3, n_4 \dots n_m\}$ denote the set of servers (worker nodes) on which containers run, and $C = \{c_1, c_2, c_3, c_4 \dots c_n\}$ denote the set of containers running on worker nodes. $c_i \in C$, $n_j \in N$, and satisfies the relation $C \subset N$. T_{c_i} denotes the size of the resource time window for container c_i . $T_{c_i} \leq T^{max} = 60$. Only resource usage within the resource time window is considered to determine the scale-up size. Based on our empirical observation, setting T^{max} to 60 was judged to be reasonable and effective. CPU_{c_i} and M_{c_i} represent the CPU and memory metrics for container c_i . Meanwhile, NPM_{c_i} denotes the memory information between n_j and c_i placed in n_j . The rest of the notation is explained in Section 4.

Table 1: Notation and Description

Notation	Description
N	The set of worker nodes n_j running containers
C	The set of containers c_i running on worker nodes
T_{c_i}	Size of the time window of container c_i (up to 60sec)
CPU_{c_i}	CPU information of container c_i
M_{c_i}	Memory information of container c_i
NPM_{c_i}	Memory information between container c_i and node n_j
R_{c_i}	Resource metrics of container c_i
$FEFP_{c_i}$	"First executed first priority" metric of container c_i
$Penalty_{c_i}$	Penalty granted to container c_i (rely on number of scale-up)
$Reward_{c_i}$	Reward granted to container c_i (rely on pause time)
PS_{c_i}	Priority score of container c_i

3.2 Requirements of Vertical Memory Elasticity

To effectively achieve vertical memory elasticity, a vertical scaling solution meet the following requirements:

3.2.1 Selecting which containers to scale. The servers (nodes) that host containers have limited resources, so if multiple applications require container scaling at the same time, not all containers can be scaled. Therefore, we need to decide which containers should be scaled and manage the scaling process to ensure that the remaining containers are not scaled.

3.2.2 Handling containers due to memory pressure. Due to the size limitations of a node's memory, some tasks must be stopped when their memory requirements exceed the available physical memory. However, since stopped tasks generally need to be reinitiated from the beginning, a method to avoid this necessity should be devised.

3.2.3 Scaling size of memory. Implementing a fixed-size memory elasticity that does not consider the resource usage trends of tasks may result in unnecessary triggering of the scale-up mechanism multiple times. Therefore, a mechanism should be included to dynamically determine the memory scale size based on per-task resource usage and usage trends.

4 DEFINE TASK PRIORITY AND SCALE SIZE

Vertical elasticity cannot infinitely allocate memory to all running tasks due to limitations in host server capacity. We model the priority of each task and scale the resources of tasks in order of priority. The size of the resource scaling is determined dynamically based on the memory usage and allocation trend of the task.

4.1 Task Priority

4.1.1 Resource Metrics

When prioritizing containers, the CPU usage of each container is a crucial factor that cannot be ignored. Tasks with high CPU usage need to be processed first. Considering only momentary CPU usage does not reflect the recent usage trend of a container. Therefore, we use the average CPU utilization over a time window T seconds to prioritize containers. For a container c_i and node n_j , $CPU_{c_i}^{Util_T}$ in Equation (1) shows the average CPU utilization of c_i over T seconds. $CPU_{c_i}^{Util_t_k}$ means CPU utilization for unit time t_k :

$$CPU_{c_i}^{Util_T} = \sum_{t_k \in T_{c_i}} \frac{CPU_{c_i}^{Util_t_k}}{|T_{c_i}|}, CPU_{c_i}^{Util_t_k} = \frac{c_i^{CPUcores_t_k}}{n_j^{CPUcores_t_k}} \quad (1)$$

where $c_i^{CPUcores_t_k}$ and $n_j^{CPUcores_t_k}$ represent the number of CPU cores allocated to containers and nodes, respectively, at each t_k . The memory usage of each container is also a factor in prioritization that cannot be ignored. Containers with high memory usage (nearly reaching the maximum memory) should be prioritized for memory scale to prevent OOM errors. Similar to CPU usage, the average memory utilization over a time window of T seconds is used to prioritize containers. For a container c_i , the average of container memory utilization over T seconds ($M_{c_i}^{Util_T}$) is defined as shown in Equation (2). $M_{c_i}^{Util_t_k}$ means memory utilization for unit time t_k :

$$M_{c_i}^{Util_T} = \sum_{t_k \in T_{c_i}} \frac{M_{c_i}^{Util_t_k}}{|T_{c_i}|}, M_{c_i}^{Util_t_k} = \frac{c_i^{MemUsed_t_k}}{c_i^{MemLimit}} \quad (2)$$

where $c_i^{MemLimit}$ is the upper limit of the container memory, and its value changes as memory undergoes vertical scale-up and scale-down. This means that $c_i^{MemLimit}$ is initially user-defined, but its value is dynamically changed by the EVMM. $c_i^{MemUsed_t_k}$ is the amount of memory being used by the container at each t_k . Containers that use a lot of memory on a node cause a large overhead compared to containers that use a small amount of memory when they are stopped and restarted by OOM or EVMM. Therefore, these tasks should be given high priority. For a container c_i and node n_j , Equation (3) shows the average memory utilization between n_j and c_i is assigned ($NPM_{c_i}^{Util_T}$). $NPM_{c_i}^{Util_t_k}$ means memory utilization between n_j and c_i for every unit time t_k :

$$NPM_{c_i}^{Util_T} = \sum_{t_k \in T_{c_i}} \frac{NPM_{c_i}^{Util_t_k}}{|T_{c_i}|}, NPM_{c_i}^{Util_t_k} = \frac{c_i^{MemUsed_t_k}}{n_j^{MemSize}} \quad (3)$$

where $n_j^{MemSize}$ represents the memory size of the node. Unlike $c_i^{MemLimit}$, $n_j^{MemSize}$ is system-defined and does not change. The

variance in memory usage should also be considered. High variance indicates high memory volatility, meaning that memory usage is likely to spike suddenly, which can cause the container to restart. Equation (4) illustrates the variance in memory utilization over T seconds for a container c_i ($M_{c_i}^{Var_T}$):

$$M_{c_i}^{Var_T} = \sum_{t_k \in T_{c_i}} \frac{[M_{c_i}^{Util_T} - M_{c_i}^{Util_t_k}]^2}{|T_{c_i}|} \quad (4)$$

We can synthesize the elements to define four components that constitute resource metrics. The importance of these four factors in constructing the resource metric varies from case to case, so we use a weight vector $W^T = (w^1, w^2, w^3, w^4)$ to ensure the derivation of the appropriate resource metric. For a container c_i , the resource metric R_{c_i} is denoted by Equation (5):

$$R_{c_i} = [CPU_{c_i}^{Util_T}, M_{c_i}^{Util_T}, NPM_{c_i}^{Util_T}, M_{c_i}^{Var_T}] * W^T \quad (5)$$

4.1.2 Task Priority Score

Kubernetes keeps a record of the execution order of each container. In this paper, we define the First Executed First Priority (FEFP) metric, which assigns high priority to containers that are executed first. This approach aligns with the first-come, first-served policy widely adopted in cloud resource provisioning [28]. Equation (6) represents the $FEFP_{c_i}$ metric for a container c_i :

$$FEFP_{c_i} = \frac{1}{c_i^{Timestamp}} \quad (6)$$

To assign higher priority to containers that run first, we define the $FEFP$ of container c_i as the reciprocal of its timestamp $c_i^{Timestamp}$. However, relying solely on $FEFP$ can result in a fixed priority, potentially leading to some tasks being consistently deprived of resource allocation. To mitigate this issue, we introduce a *Penalty* and *Reward* policy for prioritization. The more vertical scaling a container undergoes, the greater the *Penalty* it incurs. This prevents any container from monopolizing scale opportunities. For a container c_i and a node n_j , $Penalty_{c_i}$ is set higher for containers that have experienced more scale-ups compared to those that have experienced fewer scale-ups. This is defined as in Equation (7):

$$Penalty_{c_i} = 1 - \frac{c_i^{Scale}}{n_j^{Scale}} \quad (7)$$

where n_j^{Scale} represents the total number of scale-up operations for all containers running on that node, and the c_i^{Scale} is the number of scale-up incurred by c_i . On the other hand, *Reward* is given in the order of containers with the most pauses due to low priority. This avoids starvation of low-priority tasks. For a container c_i , $Reward_{c_i}$ is set such that the highest priority is the one that has experienced the most CPU limits in total execution time of c_i , as shown in Equation (8):

$$Reward_{c_i} = \frac{c_i^{PauseTime}}{c_i^{ExecutionTime}} \quad (8)$$

where $c_i^{ExecutionTime}$ represents the execution time of c_i , and $c_i^{PauseTime}$ is the duration that c_i was paused by EVMM. We can synthesize everything from sub-section 4.1 to derive the task priority score for c_i . For a container c_i , the priority score PS_{c_i} is represented as shown in Equation (9):

$$PS_{c_i} = [R_{c_i}, FEFP_{c_i}, Penalty_{c_i}, Reward_{c_i}] * W^T \quad (9)$$

where $W^T = (w^1, w^2, w^3, w^4)$ is a vector representing the weight assigned to each metric. As with R_{c_i} , the four metrics are of different importance, so we use a weight vector.

Algorithm 1 Elastic Vertical Memory Management (EVMM)

```

1: Input:  $C_{run}$ ,  $C_{pause}$ ,  $C_{remove}$ ,  $C_{isCheckpoint}$ 
2: for  $\forall c \in (C_{run} \cup C_{pause})$  do //DecisionScaleDown
3:   if  $T_{c_i} > 10$  and  $M_{c_i}^{Util} < Thr_{ScaleDown}$  then
4:     calculate  $scaledown_{c_i}$ 
5:      $ScaleDown(c_i, scaledown_{c_i})$ 
6:   end if
7: end for
8: for  $\forall c \in C_{run}$  do //Append ScaleCandidate & Delete Pause
9:   if  $M_{c_i}^{Util} > Thr_{ScaleUp}$  then
10:    calculate  $scaleup_c$  and append  $c_i$  to  $C_{scaleCandidate}$ 
11:   else if  $M_{c_i}^{Util} \leq Thr_{ScaleUp}$  and  $c_i \in C_{pause}$  then
12:      $ContinueContainer(c_i)$ 
13:     remove  $c_i$  from  $C_{pause}$ 
14:   end if
15: end for
16:  $DecisionScaleUp(C_{scaleCandidate}, C_{pause})$ 
17:  $DecisionRemove(C_{run}, C_{pause}, C_{isCheckpoint})$ 
18: for  $\forall c \in C_{remove}$  do //DecisionRepairContainer
19:   if  $n_j^{MemUsage} + c_i^{MemLimit} + scaleup_{c_i} > n_j^{MemSize}$  then
20:     break the loop
21:   else
22:      $RepairContainer(c_i, scaleup_{c_i})$ 
23:     remove  $c_i$  from  $C_{remove}$ 
24:   end if
25: end for

```

4.2 Scale Size

The EVMM does not perform scale-down and scale-up at the same rate, but dynamically adjusts reclamation and allocation based on the memory usage and allocation trends of the corresponding operations. Equation (10) represents the amount of memory reclaimed by the system when container c_i performs a scale-down. $M_{c_i}^{UtilTarget}$ means the target memory utilization of container c_i :

$$Scaledown_{c_i} = c_i^{MemLimit} - \frac{c_i^{MemUsed}}{M_{c_i}^{UtilTarget}} \quad (10)$$

This means that after performing scale-down, $M_{c_i}^{Util}$ is guaranteed to be $M_{c_i}^{UtilTarget}$. On the other hand, the size of scale-up is highly dependent on the size of the time window and the memory allocation trend. The amount of memory that should be allocated to the system when container c performs a scale-up is defined as shown in Equation (11):

$$Scaleup_{c_i} = \sum_{t=1}^{\frac{T_{c_i}+1}{10}} f(t) \quad f(t) = \begin{cases} 0, & g(t) < 0 \\ \frac{T^{max}}{10at} * g(t), & g(t) \geq 0 \end{cases} \quad (11)$$

where

$$g(t) = c_i^{MemUsed}_{10(t-1)+1} - c_i^{MemUsed}_{10t} \quad (12)$$

where the notations such as T_{c_i} and T^{max} were discussed in Section 3.1. Equation (12) shows the average memory usage for each bin of c_i . The size of the bin is set to 10 seconds. For example, if $t = 1$, it returns the average memory usage from 1 second ago to 10 seconds ago. If the value is negative, it means that the memory usage decreased in that bin. In this paper, we do not use intervals with negative average memory usage to calculate the size of the scale-up.

5 ELASTIC VERTICAL MEMORY MANAGEMENT

In this section, C_{run} represents containers running without a CPU limit from the set $C = \{c_1, c_2, c_3, c_4, \dots, c_n\}$. C_{pause} includes containers

Algorithm 2 DecisionScaleUp()

```

1: Function  $DecisionScaleUp(C_{scaleCandidate}, C_{pause})$ 
2:   for  $\forall c \in C_{scaleCandidate}$  then
3:     if  $n_j^{MemUsage} + scaleup_{c_i} > n_j^{MemSize}$  then
4:       break the loop
5:     else
6:        $n_j^{MemUsage} = n_j^{MemUsage} + scaleup_{c_i}$ 
7:        $ScaleUp(c_i, scaleup_{c_i})$ 
8:       remove  $c_i$  from  $C_{scaleCandidate}$ 
9:       if  $c_i \in C_{pause}$  then
10:         $ContinueContainer(c_i)$ 
11:        remove  $c_i$  from  $C_{pause}$ 
12:       end if
13:     end if
14:   end for
15:   for  $\forall c \in C_{scaleCandidate}$  then
16:      $PauseContainer(c_i, Quota_{pause})$ 
17:     append  $c_i$  to  $C_{pause}$ 
18:   end for

```

that requested but couldn't achieve a scale-up due to insufficient memory. C_{remove} refers to containers terminated due to memory pressure, and $C_{isCheckpoint}$ indicates containers that have been checkpointed. $C_{scaleCandidate}$ lists the candidate containers that should undergo a scale-up. Pause the container by reducing its CPU usage to minimize additional memory allocation for containers under memory pressure. It is assumed that C_{run} , C_{pause} , and $C_{scaleCandidate}$ are ordered in descending order based on PS_c , from Equation (9). For containers with tied priority scores, one is randomly selected to determine the order. For containers with tied priority scores, one is chosen at random to determine the order. T_c is initialized to 0 after each scale event. Updates to container lists, such as C_{pause} , are assumed to be globally synchronized.

Algorithm 1 describes the overall procedure for EVMM. The algorithm is executed on each node $n \in N$ respectively. First, it determines whether to scale-down based on the resource utilization of container c_i (Lines 3-8). If the memory utilization of c_i is less than $Thr_{ScaleDown}$, then scale-down is performed. The size of the scale-down is determined by Equation (10). To prevent scale-down from occurring immediately after scale-up, if T_{c_i} is too small (i.e. $T_{c_i} < 10$), scale-down will not occur. Next, consider the case where current memory utilization of c_i exceeds $Thr_{ScaleUp}$. For each exceeded container, we compute the required scale-up size (Equation (11)). Conversely, if container c_i is paused but its memory usage improves, the container is unpaused ($ContinueContainer(c_i)$) (Lines 9-16). Depending on the system's resource availability, it may not be possible to provide a scale-up to all scale candidates, so an additional algorithm is needed to select which containers to scale (described in Algorithm 2). The algorithm returns a set C_{pause} of containers that failed to scale-up and were paused due to insufficient resources. If a container needs to be stopped because the resource on each node is not available, an additional algorithm is required because we cannot arbitrarily stop any of the containers in C_{pause} (described in Algorithm 3). Finally, lines 19-26 mean the logic for deciding whether to repair the removed container. If the removed container c_i can be repaired due to memory pressure on the node, c_i is restarted from the checkpoint where it was stopped and performs a scale-up. Note that this is already stopped with the container running out of resources, so if we do not perform a scale-up it, it may be OOMKilled as soon as it is repaired.

Algorithm 2 determines whether each element in $C_{scaleCandidate}$ is eligible for a scale-up and performs a scale-up on the eligible containers based on Equation (11). If a container in C_{pause} is

Algorithm 3 *DecisionRemove()*

```

1: Function DecisionRemove ( $C_{run}$ ,  $C_{pause}$ ,  $C_{isCheckpoint}$ )
2:   for  $\forall c \in C_{pause}$  do // DecisionCheckpoint
3:     if  $M_{c_i}^{util} > Thr_{Checkpoint}$  then
4:       if  $c_i \notin C_{isCheckpoint}$  or  $T_{c_i}^{Checkpoint} > Thr_{ReCheckpoint}$  then
5:         checkpointContainer( $c_i$ )
6:         append or update  $c_i$  to  $C_{isCheckpoint}$ 
7:       end if
8:     end if
9:   end for
10:  for  $\forall c \in C_{run}$  do // DecisionRemoveRestartContainer
11:    if  $c_i$  is restarted and  $c_i \in C_{isCheckpoint}$  then
12:      append  $c_i$  to  $C_{remove}$ 
13:    end if
14:  end for
15:  for  $\forall c \in C_{isCheckpoint} \cap C_{pause}$  do // DecisionRemoveContainer
16:    if  $n_j^{MemUsage} > Thr_{ContainerStop}$  then
17:      removeContainer( $c_i$ )
18:      remove  $c_i$  from  $C_{pause}$  and append to  $C_{remove}$ 
19:    end if
20:  end for
21:  while  $n(C_{isCheckpoint}) \neq 0$  and  $n(C_{run}) < n(C_{pause})$  then
22:    Select one of the intersections of  $C_{isCheckpoint}$  and  $C_{pause}$  as  $c_i$ .
23:    removeContainer( $c_i$ )
24:    remove  $c_i$  from  $C_{pause}$  and append to  $C_{remove}$ 
25:  end while

```

eligible for a scale-up, the algorithm preforms the scale-up on that container and unpauses it (Lines 2-14). For containers that cannot undergo a scale-up because the node is running out of resources, pause them (set the CPU quota to $Quota_{Pause}$) to minimize memory allocation (Lines 15-18).

Algorithm 3 checkpoints a paused container and decides whether to remove it. First, it performs a checkpoint on container c_i that has crossed the checkpoint threshold, provided it has not already been checkpointed ($T_{c_i}^{Checkpoint}$ is the time of the last checkpoint of c_i) or has not been checkpointed recently (Lines 2-9). Meanwhile, during the course of a workload, a container may be restarted due to scaling failures, memory pressure, or other issues. When a restart occurs, the task must be reinitiated from the beginning. However, if the checkpoint data is available, restoring to a checkpoint can reduce the task's running time. Line 10-14 explain this process. Next, we can only remove containers that have a checkpoint, so lines 15 through 20, which determine whether to remove a container, are repeated only for the intersection of $C_{isCheckpoint}$ and C_{pause} . First, if the memory usage exceeds $Thr_{ContainerStop}$, it stops the execution of containers in $C_{isCheckpoint}$ until the situation is improved. This frees up memory on the node so that memory can be allocated to other containers in $C_{scalecandidate}$. Finally, we provide handling for when paused containers become more numerous than unpaused containers (Lines 21-25). As the number of paused containers increases, the allocation/repair of resources for each container becomes slower. This is similar to the deadlock problem in the system. To solve this problem, we remove one of the containers that form the intersection of $C_{isCheckpoint}$ and C_{pause} . This means that even if none of the checkpointed containers have reached $Thr_{ContainerStop}$, we remove them to improve the deadlock.

6 IMPLEMENTATION

The implementation of EVMM requires the ability to monitor and control the resources of containers running on each node. For this

reason, we implemented a module to control the resources of containers via the Container Runtime Interface (CRI) API [29]. The CRI API is an internal API used by Kubernetes to communicate with the container runtime. Additionally, to implement such a management module, Kubernetes must be able to dynamically change the memory size of a container and support container checkpointing. Starting with Kubernetes v1.25 and later, an in-place resource resize API has been implemented in the CRI API, and the CRI-O [30] runtime environment has started supporting Linux's Checkpoint/Restore In Userspace (CRIU) [31] checkpointing feature in alpha. We chose CRI-O as our container runtime because it is the only runtime that integrates checkpointing into Kubernetes. The Kubernetes and CRI-O we used is v1.27, which supports both in-place resource resize and container checkpoints.

EVMM is implemented in the Go programming language. The program runs on each worker node and manages container memory on those nodes. Although the algorithm described in Section 5 processes containers sequentially for checkpointing and repair, these operations can be time-consuming (as mentioned in Section 7). If the program were to wait for each checkpointing and repair operation to complete before managing other containers, it would be unable to handle additional container management tasks during that time. Therefore, the actual implementation utilizes Go routines to handle checkpointing and repair operations asynchronously in separate threads. Additionally, it is designed to manage multiple containers simultaneously, handling requests for scale-up/scale-down, pause/continue operations in parallel, enabling immediate responses to resource allocation and repair requests.

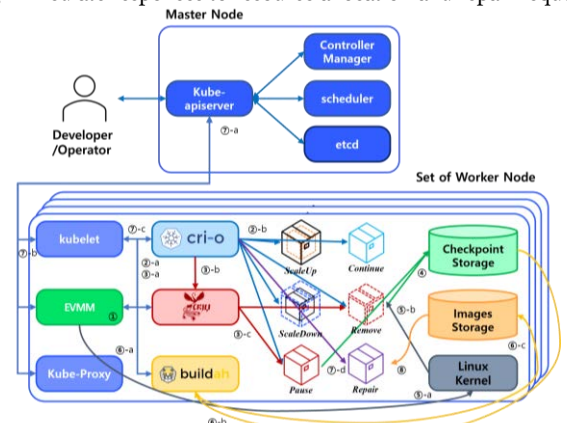


Figure 1: EVMM Architecture overview.

Figure 1 illustrates the system structure and interaction with EVMM. First, the EVMM creates a CRI RPC client to communicate with CRI-O (①). Then, in ②-a, it communicates with CRI-O to retrieve metadata and resource usage for each container in each cycle. Based on this data, EVMM makes decisions and communicates them to CRI-O. CRI-O then elastically adjusts resources (e.g., scale-up) as instructed by EVMM (②-b). If a container requires a checkpoint, EVMM requests CRI-O to perform the checkpoint, which CRI-O accomplishes using CRIU (③). The checkpoint data is then saved to storage (④). On the one hand, when a container is removed via the Kubernetes API or the CRI API, CRI-O has a 30-second timeout to complete the removal. If the timeout is reached, the container is forcibly removed. During this time, CRI-O remains busy and unresponsive to EVMM's requests. To prevent this, we use Linux's *sigkill* to remove containers at the process level (⑤), avoiding CRI-O's unresponsiveness. Finally, when repairing a container in Kubernetes, the checkpoint must be created as a container image and deployed as a Pod. EVMM uses buildah to

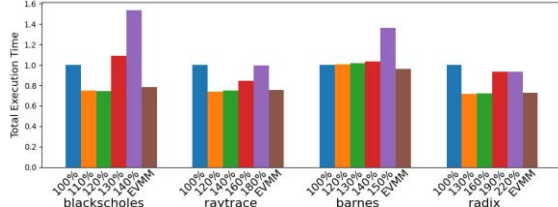


Figure 2: Total execution time for all workloads.

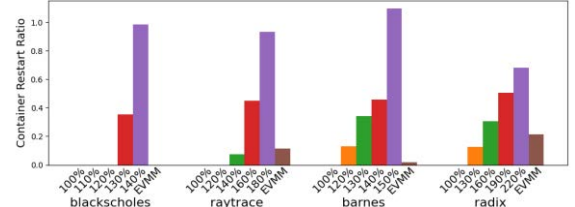


Figure 3: Container restart ratio for all workloads.

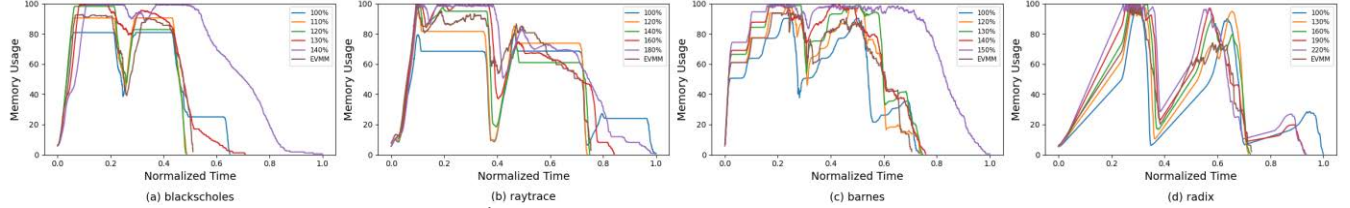


Figure 4: Node's Memory resource utilization for all workloads.

convert checkpoint data into a container image (⑥). It then requests the kube-apiserver to deploy that image as a Pod (⑦-a). The kube-apiserver, via kubelet, instructs CRI-O to create the pod and container (⑦-b & ⑦-c). Recognizing the container image as a checkpoint image, CRI-O fetches the checkpoint data (referenced by that container image) (⑧) and restores the container (⑦-d). After restoration, the container continues running as if the checkpoint was never used.

7 EVALUATION

7.1 Evaluation

7.1.1 Cluster and Environment Setup. In this experiment, a cluster consists of four nodes, one of which acts as a master node that performs cluster control. The remaining three nodes are configured as worker nodes. All nodes are configured with Intel i9-10900K processors, Ubuntu 20.04 is installed as the operating system, and they are interconnected by 1 Gbps Ethernet. The worker nodes have 32 GB of memory, and the master node has 16 GB of memory.

7.1.2 Experiment Workload. To verify the performance of EVMM experimentally, we need a memory-elastic workload. Therefore, we use the Princeton Application Repository for Shared-Memory Computers (PARSEC) [32] and the SPLASH-2x benchmark [33]. These benchmarks provide test data and programs that cover a variety of real-world applications. Table 2 describes the benchmarks used in our experiments. Memory volatility is calculated as the peak memory usage of a benchmark container divided by its average memory usage. A higher value indicates greater memory volatility for that benchmark.

7.1.3 Experiment Method. Table 3 presents the parameters used in the experiments. These parameters were selected heuristically, and there may be more optimized configurations to improve the execution time of the benchmarks. This study assumes a scenario where the memory of a container application with high memory volatility is being controlled. Each experimental workload in Table 2 consists of multiple containers, each performing a computation and terminating upon completion. In this scenario, containers do not communicate with one another, and no external requests are injected. Each container is deployed to Kubernetes at regular intervals, which is intended to give different containers different values for the FEP metric in Equation (6). In this experiment, EVMM ensures that each node has sufficient resources before running a container, preventing all containers from running simultaneously. If a node lacks sufficient resources, the container

Table 2: Workload descriptions used in the experiment.

Workload	Description	Volatility
Blackscholes	Financial analysis application	1.16
Raytrace	3D rendering applications	1.68
Barnes	N-body simulation for astrophysics	1.37
Radix	Radix sort, a numeric sorting algorithm	2.65

Table 3: Parameter configuration.

Parameter	Value
<i>QuotaPause</i>	20millicore (0.02 CPU)
<i>ThrScaleUp</i>	80%
<i>ThrScaleDown</i>	70%
<i>ThrCheckpoint</i>	82%
<i>ThrReCheckPoint</i>	120sec
<i>ThrContainerStop</i>	95%

remains in a pending state and only starts running on the node once resources become available. The performance of EVMM is evaluated by executing the workloads listed in Table 2 individually.

It is difficult to use Kubernetes VPA for a comparison experiment because it requires restarting containers every time a scale operation occurs. Therefore, in this paper, we chose an over-subscription environment for comparison. Over-subscription refers to allocating resources to more users or processes than the system can serve. In practice, it is used as a strategy to maintain high resource utilization or to serve more users even when resources are scarce, and it is widely used in cloud environments [18].

7.2 Experimental Results

Figure 2 shows the normalized total execution time of the test workloads listed in Table 2. The baseline for normalization is a standard Kubernetes environment without over-subscription. The total execution time is defined as the difference between the start time of the first container and the end time of the last container. Due to varying memory volatility and workload characteristics, comparing workloads using the same over-subscription ratio is challenging. Therefore, the figure presents different over-subscription ratios for each workload. Figure 3 shows the percentage of container restarts for each workload, calculated as the number of container restarts divided by the total number of containers in that workload. For instance, if a workload with 50 containers experiences 5 restarts, the restart rate is 0.1. Figure 4 provides a normalized graph of memory resource usage for each workload, with normalization based on the longest runtime.

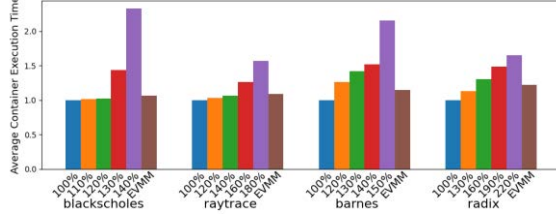


Figure 5: Average container execution time for all workloads.

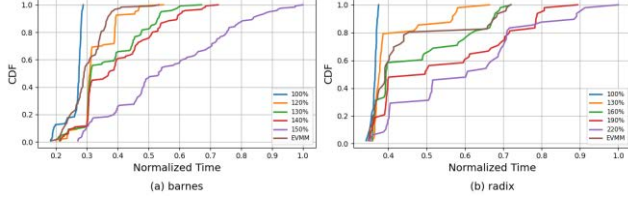


Figure 7: CDFs of average container execution time for selected workloads.

In Figure 2, EVMM demonstrates strong performance across workloads with varying memory volatility. It reduces execution time by up to 35% compared to a typical Kubernetes environment without oversubscription. Even when compared to the optimal oversubscription scenario, EVMM incurs a performance delay of only 5% or less. This delay can be attributed to the CPU usage of some containers being throttled or paused when there is memory pressure on the node, which increases the execution time of those containers (as illustrated in Figure 5). Nevertheless, the difference from the optimal oversubscription scenario remains minimal. In the case of the *barnes*, the optimal scenario is the no oversubscription case, and EVMM outperforms it by about 5%. As shown in Figure 3, EVMM can minimize container restarts even when memory resources are scarce. For the *barnes*, there were very few restarts, with a reduction of more than 70x compared to the heavily oversubscribed environment (150%). In the case of *blackscholes*, no restarts occurred at all. This suggests that *barnes* and *blackscholes* have relatively low memory volatility, making CPU throttling effective in pausing these workloads. However, for workloads with very high memory volatility, such as *radix*, even if you attempt to pause the container with CPU throttling, memory may continue to be allocated, causing the container to exceed its maximum memory, resulting in a somewhat higher restart rate. Despite this, EVMM still effectively controls container restarts.

In Figure 4, we can see that memory utilization is low for all workloads when there is no oversubscription. However, with oversubscription, memory utilization increases, but excessive oversubscription results in a significant rise in the number of container restarts (Figure 3), which in turn increases the total execution time (Figure 2). EVMM, on the other hand, aims to maintain memory utilization below the $Thr_{ContainerStop}$ for all workloads. When memory pressure occurs, EVMM stops some of the checkpointed containers, which frees up memory. This freed-up memory is then allocated to the remaining paused containers, and new containers are scheduled on those nodes. As a result, we achieve stable memory utilization below the $Thr_{ContainerStop}$ while reducing the restart rate of containers compared to an oversubscribed environment, and still attain up to 1.5x better memory utilization compared to a situation without oversubscription. This is illustrated in Figures 4(a) and 4(c). For example, *blackscholes* and *barnes* show memory utilization oscillating near $Thr_{ContainerStop}$ in the 0-0.2 and 0.2-0.4 ranges, respectively. During these intervals, some containers are checkpointed and removed. As

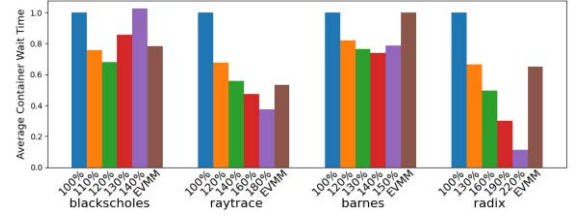


Figure 6: average container wait time for all workloads.

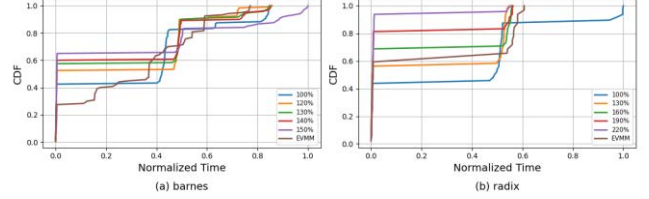


Figure 8: CDFs of average container wait time for selected workloads.

the memory pressure improves, the removed containers are restarted (in the 0.2-0.4 and 0.4-0.6 ranges, respectively). *Raytrace* and *radix* exhibit higher memory volatility compared to containers may restart beyond $Thr_{ContainerStop}$. Notably, *radix* does not oscillate at $Thr_{ContainerStop}$ due to this high volatility. Although both *raytrace* and *radix* have high memory volatility, their graphs show different behaviors. This difference arises because the unit container in *radix* is relatively large (approximately 2.5 times the maximum memory usage of *raytrace*), resulting in a significant decrease in memory after container removal (in the 0.3-0.4 range). Moreover, given that the checkpoint size is relatively large (see Table 4), the memory increase during repair is considerable in comparison to other workloads.

Figures 5 and 6 illustrate the normalized average execution time and wait time per container for all test workloads. Container execution time is defined as the sum of the time the container ran and the *CrashLoopBackOff* time, with the exclusion of the time the container was stopped by EVMM. The wait time represents the period during which the container was awaiting execution, inclusive of the interval during which EVMM terminated the container. The time a container is paused is considered part of the running time and is therefore included in the container execution time. The baseline for normalization is a standard Kubernetes environment without oversubscription. Figures 7 and 8 illustrate the cumulative distribution functions (CDFs) for container execution time and wait time for select workloads. Normalization was based on the largest value observed for each workload.

In Figure 5, EVMM demonstrates good execution times, though it is somewhat slower compared to the scenario with the optimal total execution time. This is also reflected in Figure 7, where we see that in the *barnes*, some containers experience delays due to pausing. Conversely, the *radix* shows a greater number of containers with extended execution times than *barnes*, which is attributed to the high restart rate of containers, leading to a higher proportion of containers running for longer durations. Figure 6 indicates that the wait time is similar to or slightly higher than in the optimal total execution time scenario. This is because EVMM stops the container, a new container is allocated, but the wait time reduction is amortized because the removed container must also wait to be repaired. Notably, *barnes* has the longest wait time. Since *barnes* is not memory elastic, containers removed by EVMM must wait for other containers to complete their work (as shown in Figure 8). This increases the wait time, but the total execution time is the fastest for EVMM, which is due to the effectiveness of checkpoints.

Table 4: Overhead of checkpoint and create images.

Workload	Checkpoint(s)	CreateImages(s)	Size(GB)
Blackscholes	22.2	188.6	1.7
Raytrace	31.2	138	1.58
Barnes	14.7	117.95	1.6
Radix	19	112.5	3.5

Table 4 shows the average time taken for checkpoints and image creation across all workloads. Interestingly, there is no correlation between checkpoint size and duration. *Radix* had relatively short times, averaging 19 seconds, despite having checkpoints more than twice the size of those in other workloads. This is likely due to the greater impact of container runtime load on each node. For example, in *raytrace*, which has a relatively large number of segments with memory utilization higher than *ThrContainerStop* (as shown in Figure 4), the shortest checkpoint time was 7 seconds, while the longest was 68 seconds, a difference of about 10x. In contrast, *radix* showed less variation, with checkpoint times ranging from 6 to 30 seconds. Another factor is the sheer number of containers managed by the container runtime. *Blackscholes* and *raytrace* have relatively small unit container sizes, meaning many containers run on a single node. As a result, the container runtime must process scale-up operations, checkpoint requests, and other tasks from a large number of containers simultaneously, which can slow down processing and cause delays. This suggests that we should carefully determine when and how often to perform checkpoints. Additionally, converting checkpoint data into a container image can also cause significant delays: the fastest image creation time for *raytrace* was 13 seconds, while the slowest took 190 seconds. We observed that delays in checkpointing and image generation were generally worse under high-memory conditions and when these commands were executed in parallel, which some controls failed to handle.

8 CONCLUSION

We propose a new elastic memory management method, EVMM, for Kubernetes environments. To the best of our knowledge, no previous work has defined and addressed the problem of memory elasticity in a Kubernetes environment with swap memory disabled. EVMM dynamically scales the resource size of containers up and down while monitoring the memory utilization of nodes. It also pauses and stops tasks based on priority when memory pressure occurs on a node. This minimizes container restarts while ensuring memory availability for high-priority jobs. When comparing EVMM to a traditional Kubernetes environment without vertical elasticity, we found that the total execution time of container workflows was reduced by up to 35%, and memory utilization on nodes improved by 1.5x. This approach provides an opportunity for achieving memory vertical elasticity in stateful applications.

However, we identified some issues: repairing removed containers can cause resource imbalance between nodes, resulting in container skewing to certain nodes; checkpointing when a node is low on memory can overload the container runtime, causing some controls to fail; and restarting tasks with highly volatile memory cannot always be prevented by limiting CPU usage. Future work will address these issues, further refine EVMM, and integrate its capabilities into the Kubernetes VPA.

ACKNOWLEDGMENTS

This research was supported by the MSIT(Ministry of Science and ICT), Korea, under the ICAN(ICT Challenge and Advanced Network of HRD) support program(IITP-2024-RS-2022-00156439) supervised by the IITP(Institute for Information & Communications Technology Planning & Evaluation)

REFERENCES

- [1] Sethur, Amrith Rajagopal, et al. "An efficient fault tolerant workflow scheduling approach using replication heuristics and checkpointing in the cloud." *Journal of Parallel and Distributed Computing* 136 (2020): 14-28.
- [2] Das, Arun, et al. "Distributed machine learning cloud teleophthalmology IoT for predicting AMD disease progression." *Future Generation Computer Systems* 93 (2019): 486-498.
- [3] Rzdca, Krzysztof, et al. "Autopilot: workload auto-scaling at Google." *Proceedings of the Fifteenth European Conference on Computer Systems*. 2020.
- [4] Bhasi, Vivek M., et al. "Kraken: Adaptive container provisioning for deploying dynamic dags in serverless platforms." *Proceedings of the ACM Symposium on Cloud Computing*. 2021.
- [5] C. Reiss, J. Wilkes, and J. L. Hellerstein. Google cluster-usage traces: format schema. Technical report at <https://github.com/google/clusterdata/>, Google, Mountain View, CA, USA, 2011.
- [6] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *3rd ACM Symposium on Cloud Computing (SoCC'12)*, page 7. ACM, 2012.
- [7] Kwan, Anthony, et al. "Hyscale: Hybrid and network scaling of dockerized microservices in cloud data centres." *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2019.
- [8] Vu, Dinh-Dai, Minh-Ngoc Tran, and Younghun Kim. "Predictive hybrid autoscaling for containerized applications." *IEEE Access* 10 (2022): 109768-109778.
- [9] Verma, Shveta, and Anju Bala. "Auto-scaling techniques for IoT-based cloud applications: a review." *Cluster Computing* 24.3 (2021): 2425-2459.
- [10] Buchaca, David, et al. "Proactive container auto-scaling for cloud native machine learning services." *2020 IEEE 13th International Conference on Cloud Computing (CLOUD)*. IEEE, 2020.
- [11] Dogani, Javad, Reza Namvar, and Farshad Khunjush. "Auto-scaling techniques in container-based cloud and edge/fog computing: Taxonomy and survey." *Computer Communications* (2023).
- [12] <https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler>
- [13] <https://kubernetes.io/>
- [14] Rattihalli, Gourav, et al. "Exploring potential for non-disruptive vertical auto scaling and resource estimation in kubernetes." *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*. IEEE, 2019.
- [15] Lloyd, Seth. "Ultimate physical limits to computation." *Nature* 406.6799 (2000): 1047-1054.
- [16] Williams, Dan, et al. "Overdriver: Handling memory overload in an oversubscribed cloud." *ACM SIGPLAN Notices* 46.7 (2011): 205-216.
- [17] Koh, Kwangwon, et al. "Disaggregated cloud memory with elastic block management." *IEEE Transactions on Computers* 68.1 (2018): 39-52.
- [18] Luksa, Marko. *Kubernetes in action*. Simon and Schuster, 2017.
- [19] Al-Dhuraibi, Yahya, et al. "Autonomic vertical elasticity of docker containers with elasticsearch." *2017 IEEE 10th international conference on cloud computing (CLOUD)*. IEEE, 2017.
- [20] Rattihalli, Gourav, et al. "Exploring potential for non-disruptive vertical auto scaling and resource estimation in kubernetes." *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*. IEEE, 2019.
- [21] Wang, Nan, et al. "DYVERSE: dynamic vertical scaling in multi-tenant edge environments." *Future Generation Computer Systems* 108 (2020): 598-612.
- [22] Buchaca, David, et al. "Proactive container auto-scaling for cloud native machine learning services." *2020 IEEE 13th International Conference on Cloud Computing (CLOUD)*. IEEE, 2020.
- [23] Choi, Jin-young, Minkyung Cho, and Jik-Soo Kim. "Employing vertical elasticity for efficient big data processing in container-based cloud environments." *Applied Sciences* 11.13 (2021): 6200.
- [24] Zhao, Yuxuan, and Alexandru Uta. "Tiny autoscalers for tiny workloads: Dynamic CPU allocation for serverless functions." *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2022.
- [25] Jang, Hyeon-Jun, and Hyun-Wook Jin. "SlackAuto: Slack-Aware Vertical Autoscaling of CPU Resources for Serverless Computing." *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*. IEEE, 2024.
- [26] Pavlenko, Anna, et al. "Vertically Autoscaling Monolithic Applications with CaaS: Scalable Container-as-a-Service Performance Enhanced Resizing Algorithm for the Cloud." *Companion of the 2024 International Conference on Management of Data*. 2024.
- [27] Chen, Wei, et al. "Pufferfish: Container-driven elastic memory management for data-intensive applications." *Proceedings of the ACM Symposium on Cloud Computing*. 2019.
- [28] He, Sijin, et al. "Elastic application container: A lightweight approach for cloud resource provisioning." *2012 IEEE 26th International Conference on Advanced Information Networking and Applications*. IEEE, 2012.
- [29] <https://github.com/kubernetes/cri-api>
- [30] <https://github.com/cni/cni>
- [31] <https://cni.org/>
- [32] Bienia, Christian, et al. "The PARSEC benchmark suite: Characterization and architectural implications." *Proceedings of the 17th international conference on Parallel architectures and compilation techniques*. 2008.
- [33] <https://www.capsl.udel.edu/splash/index.html>